

Vectera.ai – RAG System Technical Assessment

Overview

You are asked to build a Retrieval-Augmented Generation (RAG) system over a set of investment-related documents (provided separately).

The goal is to demonstrate your ability to:

- build a working RAG pipeline
- handle real-world document complexity
- reason over multiple documents, versions, and data types
- clearly explain your design decisions and tradeoffs

This is not a production system. We are evaluating your thinking, approach, and execution.

■ Time Expectation

- Target: 6–10 hours total
- Please do not spend more than ~1–2 days

■ Input

You will be provided with ~10 documents, including:

- investor presentations (PDFs)
- multiple versions of the same company's materials
- long-form strategy decks
- third-party reports

■ Objective

Build a system that allows a user to:

Ask natural language questions about the documents and receive accurate, source-grounded answers with citations.

■ Technical Requirements

Required technologies

- **Programming language:** Python (required)
- **Database:**
- Create a free account on Snowflake (preferred)
- OR use an equivalent database service (e.g., Postgres, Supabase, etc.)

■ Your system should store and retrieve document data using this database layer.

■ Functional Requirements

Core functionality (required)

Your system should include:

- Document ingestion (PDF → text)
- Chunking
- Embedding + retrieval
- LLM-based answer generation
- **Citations (required)**
- Responses must clearly reference source documents and/or sections

Application requirement (important)

You must build a working application with a basic user interface

Examples:

- Streamlit (recommended)
- simple web app (Flask/FastAPI + frontend)

■ **CLI-only solutions are not acceptable**

Key capabilities to demonstrate

Your system should demonstrate awareness of the following:

1. Version awareness

- Some documents represent different versions of the same company
- Your system should avoid blindly mixing conflicting values across versions

2. Cross-document reasoning & conflicts

- Some documents may contain different or conflicting information
- preserve attribution (e.g., “according to document X...”)
- avoid merging conflicting facts into a single answer

3. Charts, tables, and structured content

- Documents contain charts, tables, and numerical data
- You may extract simple tables, OR
- explicitly explain limitations in handling charts/visuals

We are evaluating how you handle or reason about this, not expecting perfect extraction.

■■ **Deliverables**

1. Working application

- Simple UI (not CLI)
- User can:
- ask questions
- receive answers with citations

2. Code repository (GitHub or similar)

Include:

- clear setup instructions
- how to configure database (Snowflake or equivalent)
- environment setup
- how to run the app

3. README (important)

Please include a short write-up covering:

- System architecture (high-level)
- How you used the database (Snowflake or equivalent)
- Chunking strategy
- Retrieval approach
- How you handled:
- versioning
- conflicting information
- charts/tables
- Known limitations
- What you would improve with more time

4. Demo (5–10 minutes)

- Live walkthrough or recorded video
- Show example queries and system behavior

■ Example Questions

We may ask you to demonstrate questions such as:

- “What is [Company X]’s key strategy?”
- “How has [metric] changed across document versions?”
- “What drives demand according to these materials?”
- “Are there conflicting data points across documents?”
- “Summarize key trends shown in the documents”

■ Evaluation Criteria

1. Retrieval quality

- Are the right documents/chunks being retrieved?
- Are answers grounded in source material?

2. Handling of complexity

- Version awareness (multiple document versions)
- Handling of conflicting information

- Awareness of limitations (charts, tables, etc.)

3. System design thinking

- Clarity of architecture
- Thoughtfulness of tradeoffs
- Understanding of how this would scale

4. Code quality

- Readability
- Structure
- Simplicity vs overengineering

5. Communication

- Clarity in README and demo
- Ability to explain decisions and limitations

■ ■ Notes

You are not expected to fully solve:

- perfect chart extraction
- complete conflict resolution
- production-grade infrastructure

We are looking for:

Clear thinking, strong fundamentals, and awareness of real-world challenges

■ (Optional Consideration)

Assume documents may belong to different clients. You do not need to fully implement access control, but we are interested in how you would approach it.

■ Submission

Please provide:

- link to code repository
- README
- demo (video or live session)

Final Note

This exercise is designed to reflect real-world challenges we are working on at vectera.ai. We are less interested in a “perfect system” and more interested in how you approach ambiguity, complexity, and tradeoffs.

If you have any questions, feel free to reach out.